Blockchain Fundamentals

Everything in Phase 1, explained from nothing. No prior knowledge assumed, every technical word defined the first time it appears.

CONTENTS

1 **Money is just a list** — why removing the bank is hard

2 **The double-spending problem** — the one problem all of this solves

3 **Hash functions** — tamper-evident fingerprints

4 **Keys and signatures** — ownership without accounts

5 **What a transaction is** — putting the tools together

6 **Blocks and the chain** — why the past cannot be rewritten

7 **Nodes and gossip** — how the network actually talks

8 **Consensus** — safety, liveness, and what can go wrong

9 **Byzantine faults** — liars, and the two-thirds rule

10 **Sybil attacks** — why fake identities must be made useless

11 **Proof of Work** — Bitcoin's answer, and its price

12 **Proof of Stake** — the alternative Algorand builds on

13 **Public vs permissioned** — where Hyperledger Fabric sits

14 **Glossary** — every term in one place

15 **Build it yourself** — a working chain in 60 lines of Python

16 **Checkpoint** — questions, then answers

BEFORE YOU START

How to read this document

This covers Phase 1 of your eight-phase plan. It is the foundation everything else stands on, so it is worth going slowly. Budget about six hours.

Read it in order. Each section uses only ideas already introduced. Section 6 will not make sense without section 3, and section 11 will not make sense without section 9.

Every bold-italic term is defined where it first appears, and again in the glossary in section 14. If a word looks like jargon, it is, and it will have been explained.

Do section 15. Reading about hash chains and building one are different kinds of knowing. It is sixty lines and it will take you under an hour.

Then answer section 16 out loud, without notes. If you cannot, the section that failed you is named next to each question. Go back to it.

WHAT YOU WILL BE ABLE TO DO AFTERWARDS

Explain what a blockchain is to someone who has never heard of one, without hand-waving.

Say precisely why Bitcoin makes you wait an hour, and why Algorand does not.

Read the words *ledger, hash, signature, node, fork, consensus, Byzantine, Sybil, finality* in a research paper without pausing.

Understand why your project's consensus algorithm exists at all — which is the thing that makes Phase 3 possible.

ONE HONEST WARNING

Blockchain writing is full of hype and full of people repeating things they have not checked. This document sticks to mechanism: what the machine actually does. Where something is a trade-off rather than a triumph, it says so.

SECTION 1

Money is just a list

Forget blockchain entirely for a moment. Think about your bank.

Your money is not a physical thing in a vault with your name on it. It is a number in a database the bank owns. When you pay a friend 500 rupees, nothing moves. The bank edits two rows:

```
Swayam    10,000  ->   9,500
Friend     2,000  ->   2,500
```

That is all money is: a **ledger** — a list of who owns what, and a history of transfers. It works because everyone trusts the bank to keep the list honestly. The bank is a **central authority**: one organisation whose copy of the list is, by definition, the correct one.

> *The question that created blockchain is simply: can we keep that list without anyone being in charge of it?*

A list maintained by thousands of strangers, where no single one of them can cheat, and no single one of them can be shut down. That sounds achievable until you look at what goes wrong.

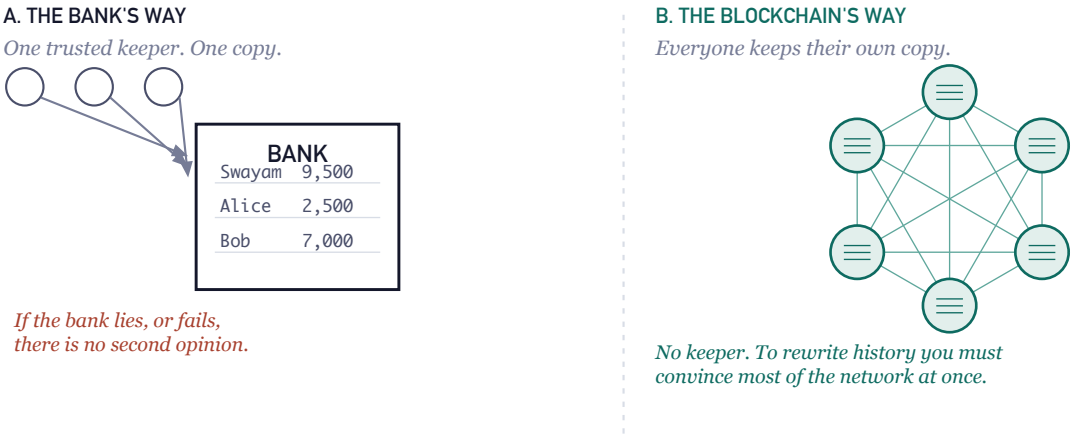


Figure 1 — The same list, kept two different ways.

WORDS DEFINED IN THIS SECTION

Ledger — a record of accounts and transfers. A list. Nothing more mysterious than that.

Central authority — a single party everyone trusts to hold the true copy. A bank, a government, a company database.

Decentralised — the opposite: no single party holds the true copy; the network collectively does.

SECTION 2

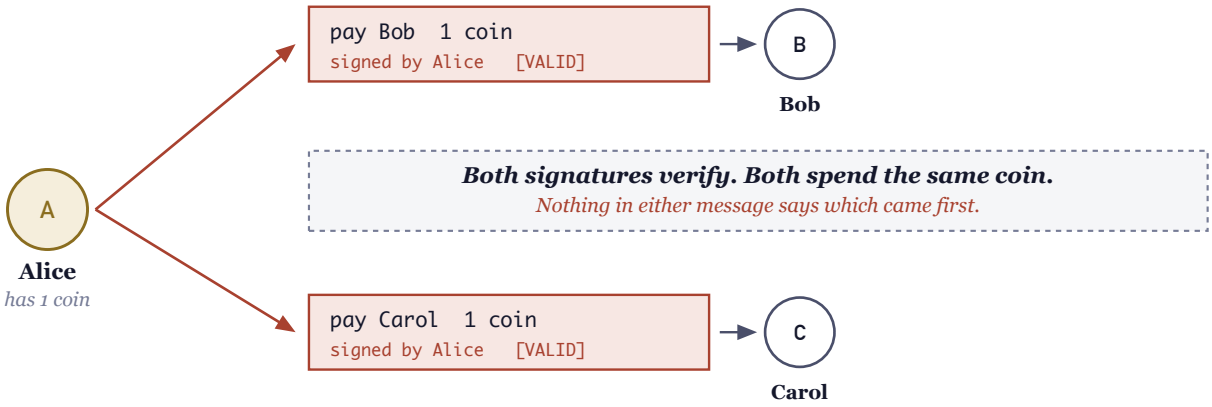
The double-spending problem

Here is what goes wrong the moment you remove the bank.

Suppose I have one coin. I send it to Alice, and at the same instant I send the same coin to Bob — announcing each to a different part of the network. Who actually has it?

With a bank this is trivial. The bank sees both requests, processes whichever arrived first, and rejects the second. **One entity, therefore one order of events.**

Without a bank it is brutal. Alice's copy of the ledger says she was paid. Bob's copy says he was. Both copies are internally consistent. Both payments are correctly signed by me. Nothing inside either message says which one I sent first — or that the other exists at all.



THE ONLY FIX IS AGREEING ON ORDER — WHICH IS WHAT CONSENSUS MEANS.

Figure 2 — One coin, two promises. Both look perfectly valid on their own.

THIS IS THE WHOLE PROBLEM

Double spending is the core problem every cryptocurrency exists to solve: spending the same money twice by telling different people different things.

Every mechanism in the rest of this document — hashes, chains, mining, committees, voting — is machinery built to answer one question: **in what order did these events happen, and who says so?**

Keep this in mind. When a later section feels arbitrary, ask how it helps establish order, and it will usually snap into focus.

SECTION 3 • TOOL ONE

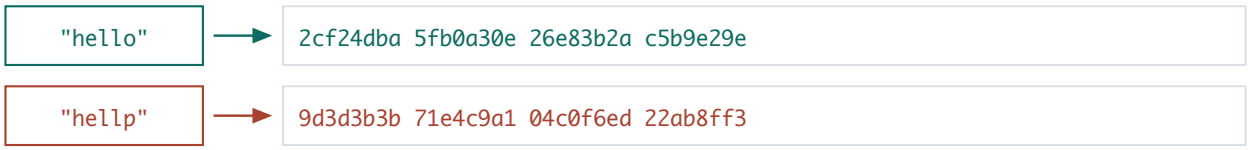
Hash functions

A hash function is a machine that eats data of any size and produces a fixed-length fingerprint.



THE AVALANCHE EFFECT

Change one letter — o becomes p — and the fingerprint is unrecognisable.



Nothing about the second output hints that the input was almost identical.

Figure 3 — A hash is a fingerprint: same input, same output; one letter different, everything different.

The four properties that matter

Deterministic	The same input always produces the same output. Every computer on earth, run a century apart, gets the same answer.
Fixed size	A one-gigabyte film and the word *hi* both produce the same 64-character output. Size in, size out, unrelated.
One-way	Given the output, you cannot work backwards to the input. Your only option is to guess inputs and check. This property is what mining is built on.
Avalanche	Change one bit of input and the entire output changes unrecognisably. There is no partial similarity, no gradient, no clue.

What it is actually for

A hash is a **tamper-evident seal**. Publish the hash of a document today; if anyone alters so much as a comma later, the hash will not match. You get a 32-byte proof that a file of any size is unchanged.

That is the property section 6 turns into a chain. Hold onto it.

SHA-256

SHA-256 is the specific hash function Bitcoin uses, and the one we use in your project. The name means *Secure Hash Algorithm, 256 bits of output*. 256 bits is 64 hexadecimal characters.

Hexadecimal (hex) just means counting in base 16, using 0-9 then a-f. It is a compact way to print raw bytes. `ff` is 255; `00` is zero.

You will see hashes written as `2cf24dba...` — that is just the first few hex characters, shown because the full 64 are unwieldy.

SECTION 4 • TOOL TWO

Public keys and digital signatures

This is the tool that makes ownership possible with no accounts, no usernames, and no passwords.

You generate a **key pair** — two mathematically linked numbers:

- **Private key** (written `sk`, for *secret key*) — an enormous secret number. Only you have it. **This is your money.** Lose it and the funds are gone permanently; there is no reset, because there is nobody to ask.
- **Public key** (written `pk`) — derived from the private key and shared freely. **This is your address.** People send you money here.

The essential asymmetry: you can compute `pk` from `sk`, but never `sk` from `pk`. The maths runs one way only.



Figure 4 — Only you can produce it. Everyone can check it. Your secret never moves.

So I can write *pay Alice 500*, sign it with my private key, and shout it across the world. Anyone can verify — using my *public* key — that the message genuinely came from whoever holds that private key, and that not a single character was altered in transit. **And I never revealed my secret.**

> **WHY THIS BEATS A HANDWRITTEN SIGNATURE**
>
> A physical signature looks the same on every document, so it can be traced and forged. A digital signature is computed *from the message itself* — sign a different message and you get a completely different signature.
>
> That means a signature cannot be lifted from one document and pasted onto another. It proves not just *who*, but *who agreed to exactly this text*.

SECTION 5

What a transaction actually is

Now we can assemble the two tools into something useful.

```
Transaction {
    from:      pk_swayam      # my public key
    to:        pk_alice       # her public key
    amount:    500
    signature: Sign(sk_swayam, everything above)
}
```

Anyone receiving this can check two things independently, with no help from anyone:

1. Is the signature valid for `pk_swayam`? That proves I authorised it.
2. Does `pk_swayam` hold at least 500 in the ledger? That proves I am not spending money I do not have.

NOTICE WHAT IS STILL MISSING

Nothing here stops me creating **two** such transactions spending the same 500. Both would be perfectly valid on their own. Both would pass both checks.

Signatures solve **authenticity** — who agreed to what. They do not solve **ordering** — what happened first.

That gap is exactly the double-spending problem of section 2, and the next four sections are about closing it.

SECTION 6

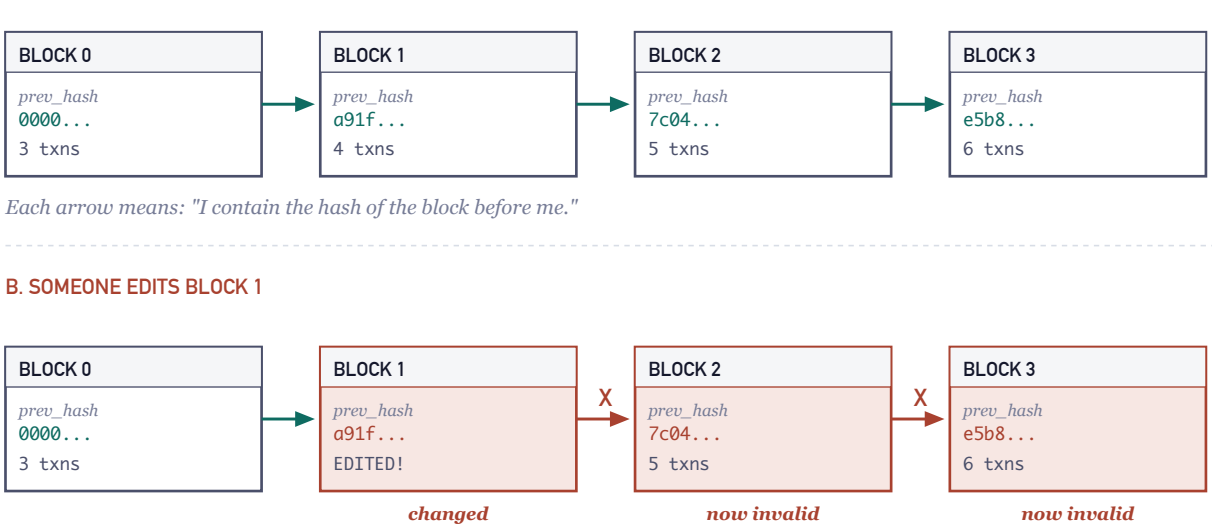
Blocks, and why it is a chain

Agreeing on transactions one at a time would be far too slow, so we batch them into a **block**.

```
BLOCK #42
  prev_hash:    <hash of block #41>    # the chain link
  round:        42
  timestamp:    2026-09-03T10:14:22Z
  transactions: [tx1, tx2, tx3, ... tx2000]
```

The critical field is `prev_hash`: every block contains the hash of the block before it. That single line is what makes the past immutable.

Suppose an attacker wants to alter a transaction back in block 1. Changing it changes block 1's hash — by the avalanche property, completely. But block 2 stores block 1's *old* hash, so block 2 is now inconsistent. Repairing block 2 changes block 2's hash, which breaks block 3. And so on, all the way to the present.



Editing block 1 changes its hash. Block 2 still stores the OLD hash, so block 2 breaks.
Fixing block 2 breaks block 3, and so on to the present day.

Figure 5 — Each block seals the one before it, so a single edit unravels everything after.

THE KEY INSIGHT

You cannot change the past without rewriting everything after it. That is the entire meaning of the word *chain* in blockchain. It is not a metaphor and it is not a database structure — it is just hashes pointing backwards.

The word **immutable** gets used for this. It does not mean the data physically cannot be edited. It means editing it is detectable by anyone, instantly, and that hiding the edit requires redoing all the work that came after.

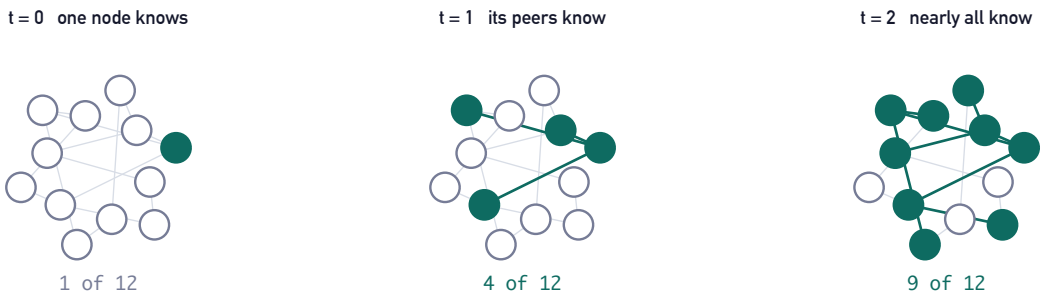
SECTION 7

The network: nodes and gossip

There is no server. There are **nodes** — computers, each holding a full copy of the blockchain and each running the same software.

They communicate by **gossip**, which works exactly like a rumour:

- Each node connects to a handful of random peers. Your project's engine uses about eight, and so did the original Algorand prototype.
- When you hear a new message, you forward it to all your peers.
- You never forward the same message twice, or it would loop forever.
- Within a few hops, the message has reached the entire network.



THIS IS ALSO WHY BLOCKCHAINS ARE SLOW: NEWS TRAVELS AT GOSSIP SPEED, NOT DATABASE SPEED.

Figure 6 — Nobody broadcasts to everyone. Each node tells a few peers, and in a few hops it has reached all.

No node knows the whole network. There is no central broadcaster. Yet everything reaches everyone. This design is why blockchains survive nodes going offline — and also **why they are slow**: information travels at gossip speed, with a delay at every hop, not at the speed of writing to a database.

A WORD YOU WILL MEET: PEER-TO-PEER

Peer-to-peer (P2P) means every participant is both a client and a server — there is no separate class of machine that serves the others. Every node asks for data and provides it.

Propagation delay is the time for a message to reach the whole network. Measurements of Bitcoin's network put roughly one second for a small message to reach 90% of peers. Your project's timeouts are chosen around this number.

SECTION 8

What consensus actually means

Consensus means: all honest nodes agree on the same ledger, in the same order, forever.

That is the whole job. It splits into two requirements which pull against each other.

PROPERTY	MEANING	IF IT BREAKS
Safety	No two honest nodes ever accept different histories.	Double-spending becomes possible. Money can be forged. **Fatal.**
Liveness	The chain keeps making progress; new blocks keep being added.	Nothing ever confirms. The system is frozen but not corrupt.

The distinction matters more than it looks. A system that stops is annoying. A system that disagrees is broken beyond repair. So when a consensus protocol must choose, **it always sacrifices liveness to preserve safety** — it would rather halt than let two versions of history exist.

You will see this choice made explicitly in your project: when Algorand cannot prove a decision is unique, it marks it *tentative* and refuses to confirm it, rather than risk being wrong.

Why it is hard: nodes that lie

If every node were honest and merely occasionally offline, this would be a solved problem with decades-old textbook answers. The difficulty is that some nodes are actively hostile. They lie, they send different messages to different peers, they stay silent at the worst moment, and they collude.

The technical term is a **Byzantine fault**, and it is the subject of the next section.

SECTION 9

Byzantine faults and the two-thirds rule

Computer scientists split failures into two kinds, and the gap between them is enormous.

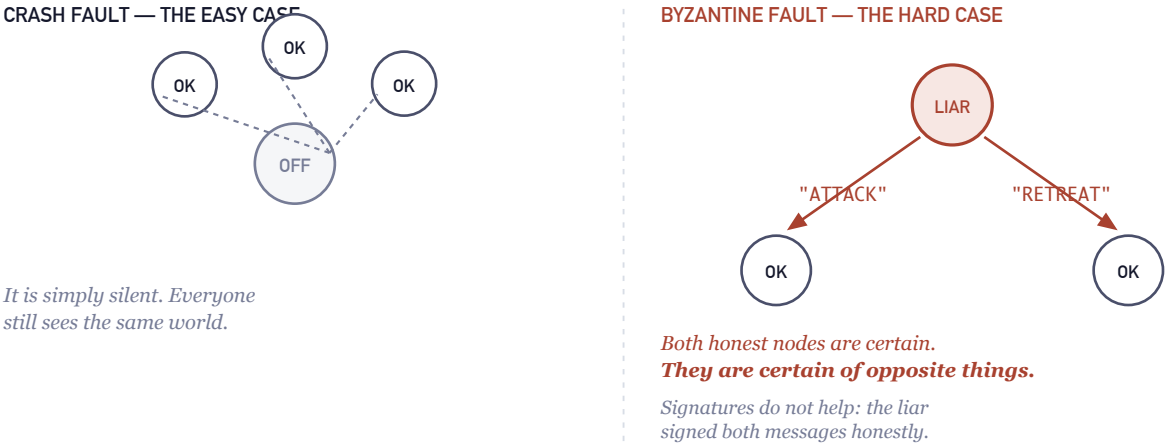
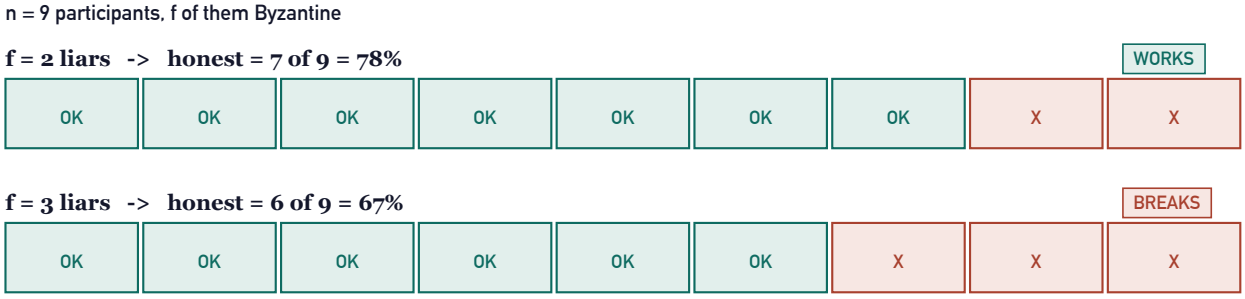


Figure 7 — A crashed node says nothing. A Byzantine node says different things to different people.

The name comes from a 1982 paper by Lamport, Shostak and Pease, which framed it as a story. Several generals surround a city. They must *all* attack or *all* retreat — a split is a catastrophe. They can only communicate by messenger. And some generals are traitors who will tell half the group to attack and the other half to retreat.

> *The proven result: agreement is possible if and only if more than two thirds of participants are honest.*



The reasoning:
you must decide without hearing from f nodes (they may be silent), so you act on n - f replies;
of those, f may be lies. For honest replies to still outnumber lies: n - 2f > f, i.e. n > 3f.

Figure 8 — Why the magic number is two thirds, and not one half.

REMEMBER THIS NUMBER

The **two-thirds** threshold appears everywhere in this field, and it will appear throughout your project — in the vote threshold T, in the honest-stake assumption h, in the committee sizing.

In Phase 3 you will derive it properly rather than take it on trust, and you will see that it is not a rule someone chose — it falls out of the arithmetic as the only region where safety and liveness can both hold.

PBFT (Practical Byzantine Fault Tolerance, 1999) is the classic protocol that achieves this. It works beautifully for perhaps 20 to 50 known servers, and collapses beyond that, because every node must message every other node.

SECTION 10

The Sybil attack

There is a hole in everything said so far, and it is a big one.

The network is open — anyone can join. So I create **one million fake identities**. Now "more than two thirds of participants are honest" is a meaningless sentence, because I *am* 99% of the participants. I vote however I like, as often as I like.

WHY THE ODD NAME

Named after a 1973 case study of a woman with multiple personality disorder, published under the pseudonym Sybil. The term entered computer science in a 2002 paper by John Douceur.

A **Sybil attack** is any attack where one party gains disproportionate influence by pretending to be many parties.

So every blockchain must answer one question before anything else works:

What makes an identity expensive?

Because identities are free, influence must be tied to something that is not. There are two established answers, and they are the subjects of the next two sections.

ANSWER	INFLUENCE IS TIED TO	USED BY
Proof of Work	electricity burned — computation you must actually perform	Bitcoin, and most early cryptocurrencies
Proof of Stake	money held — value you must actually own and risk	Algorand, Ethereum today, and **your project**

SECTION 11

Proof of Work, and the price it pays

Bitcoin's answer, from 2008: make votes cost electricity rather than identities.

To add a block you must find a number — called a **nonce**, meaning a number used once — such that the hash of the block plus that nonce is below some target, which in practice means it starts with a run of zeros:

```
SHA-256( block_contents + nonce )  <  target
```

Because hashes are one-way (section 3), there is no clever method. Your only option is to guess, trillions of times per second. This is **mining**. It is a lottery where the tickets cost real electricity, and **fake identities buy you nothing**: one computer split into a million identities has exactly the hash power of one computer. The Sybil attack is defeated by physics.

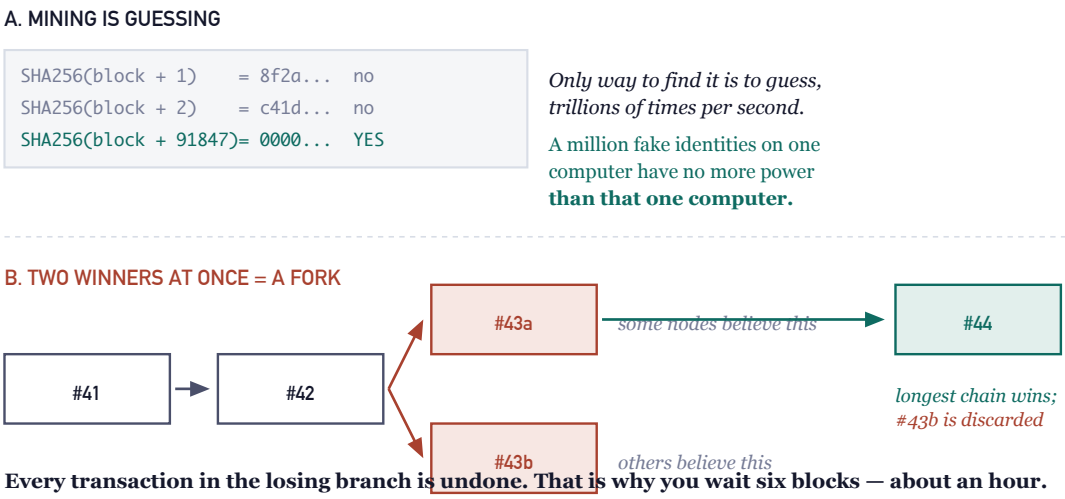


Figure 9 — Mining is a lottery with real electricity as the ticket price — and it forks.

What this costs

Forks are normal Two miners can win at nearly the same moment. Both broadcast, the network splits, and one branch is later discarded along with its transactions.

Blocks must be slow	Bitcoin targets ten minutes per block specifically so that forks stay rare enough to resolve.
Confirmation is slow	You wait about six blocks — roughly an hour — before treating a payment as settled.
Finality is only probabilistic	A transaction is never mathematically final. It just becomes exponentially unlikely to be reversed. **This is the point Algorand attacks.**
Energy	The entire network burns power continuously so that one participant wins.
Throughput	About seven transactions per second. Visa handles roughly 65,000.

SECTION 12

Proof of Stake

The alternative: make votes cost money instead of electricity.

Your influence is proportional to the coins you hold. Fake identities are again useless — splitting 1,000 coins across 100 accounts still leaves you with 1,000 coins of influence, not 100,000. The Sybil attack is defeated by arithmetic rather than physics.

The advantages are obvious: no electricity waste, and blocks can be fast because there is no puzzle to solve.

The difficulty is subtler. Producing a block is now free, so a cheater can sign two conflicting blocks at no cost whatsoever. This is called the **nothing-at-stake** problem. Most proof-of-stake systems answer it with **slashing**: misbehave provably, and your deposit is destroyed.

HOW ALGORAND DIFFERS — AND WHY THIS MATTERS TO YOU

Algorand is proof of stake, but it **never punishes anyone**. There is no slashing in the paper at all.

It does not need any, because it does not make cheating *expensive* — it makes cheating **impossible**. Stake is used only to stop an attacker amplifying their power with fake identities. As long as they hold less than a third of the money, the protocol guarantees no fork can form, so there is nothing to punish them for.

That is a genuinely different design philosophy from most proof-of-stake systems, and it is worth being able to state clearly — it is the kind of thing you will be asked about.

SECTION 13

Public, private, and where Fabric sits

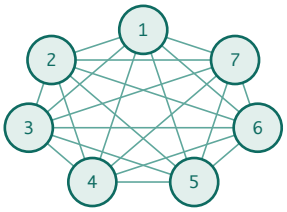
Everything so far assumed an open network anyone can join. That is one option, not the only one.

PUBLIC (Bitcoin, Ethereum)



- anyone can join
- identities unknown
- needs PoW or PoS to stop Sybils
- thousands of nodes, slow

PERMISSIONED (Hyperledger Fabric)



- membership issued by a CA
- every node has a known identity
- Sybil attack largely off the table
- a handful of nodes, fast

This is your project's world.

Figure 10 — Where your project sits: a small, permissioned network with known members.

	PUBLIC (PERMISSIONLESS)	PRIVATE (PERMISSIONED)
Who can join	Anyone, anonymously	Only approved, identified members
Examples	Bitcoin, Ethereum	Hyperledger Fabric, Corda
Sybil risk	Severe — needs PoW or PoS	Low — membership is controlled by a CA
Typical size	Thousands of nodes	5 to 100 nodes
Speed	Slow, by necessity	Fast
Trust model	Trust nobody	Members are known, but may still misbehave

What this means for your project

Hyperledger Fabric is permissioned. Every node holds an identity issued by a certificate authority, so the network layer already resists Sybil attacks — you cannot join at all without being admitted.

You might reasonably ask: if members are known and vetted, why bother with Byzantine fault tolerance at all?

THE ANSWER, AND THE JUSTIFICATION FOR YOUR WHOLE PROJECT

Known does not mean trustworthy. A permissioned member can still be hacked, bribed, coerced, or simply run buggy software. Identity tells you *who* misbehaved — it does not stop them.

Fabric 2.5 ships only **Raft** as its ordering service. Raft survives nodes that **crash**. It does not survive nodes that **lie** — one malicious leader can corrupt the log. Solo and Kafka are deprecated, and a Byzantine option is not available.

So Fabric 2.5 has **no Byzantine-fault-tolerant consensus at all**. That gap is precisely what you are filling by implementing Algorand's BA* as a custom ordering service. When someone asks why the project is worth doing, this paragraph is the answer.

SECTION 14

Glossary

Every technical term used in this document, with the section that introduces it. Come back here whenever a word stops meaning anything.

Avalanche effect	Changing one bit of a hash function's input changes the whole output. Why you cannot tell from a hash whether two inputs were similar. *Section 3*
Block	A batch of transactions plus metadata, including the hash of the previous block. *Section 6*
Byzantine fault	A node that behaves arbitrarily — lying, equivocating, colluding — rather than simply stopping. Named after the Byzantine Generals problem. *Section 9*
Central authority	A single trusted party holding the definitive copy of a record. *Section 1*
Consensus	All honest nodes agreeing on the same ledger in the same order. *Section 8*
Crash fault	A node that stops responding. Far easier to tolerate than a Byzantine fault, because it never says anything false. *Section 9*
Decentralised	No single party holds the true copy; the network collectively does. *Section 1*
Double spending	Spending the same money twice by telling different parts of the network different things. The core problem. *Section 2*
Equivocation	Sending contradictory messages to different peers. The signature characteristic of a Byzantine node. *Section 9*
Finality	The guarantee that a committed transaction can never be reversed. **Probabilistic** finality (Bitcoin) means reversal grows unlikely; **absolute** finality (Algorand) means it is impossible. *Sections 11, 12*
Fork	Two valid but incompatible versions of the chain existing at once. *Section 11*
Gossip	A protocol where each node forwards messages to a few random peers, so news spreads without any central broadcaster. *Section 7*
Hash function	A one-way function producing a fixed-size fingerprint of any input. *Section 3*
Hexadecimal	Base-16 notation using 0-9 and a-f, used to print raw bytes compactly. *Section 3*
Immutable	Not literally unchangeable, but changeable only in a way that is immediately detectable by everyone. *Section 6*
Ledger	A record of who owns what and how it moved. A list. *Section 1*
Liveness	The property that the system keeps making progress. *Section 8*
Mining	Repeatedly guessing nonces to find one satisfying the proof-of-work target. *Section 11*
Node	One computer running the blockchain software and holding a copy of the chain. *Section 7*

Nonce	A "number used once" — the value miners search for. *Section 11*
Nothing at stake	The problem that in proof of stake, signing conflicting blocks costs nothing. *Section 12*
Peer-to-peer	Every participant is both client and server; there is no privileged machine. *Section 7*
Permissioned	A network where membership is controlled and identities are known. Fabric is one. *Section 13*
Private key (sk)	The secret half of a key pair. Effectively, it is your money. *Section 4*
Proof of Stake	Sybil resistance by tying influence to money held. *Section 12*
Proof of Work	Sybil resistance by tying influence to computation performed. *Section 11*
Public key (pk)	The shareable half of a key pair. Effectively, your address. *Section 4*
Safety	The property that honest nodes never accept conflicting histories. The one you never trade away. *Section 8*
SHA-256	The specific hash function used by Bitcoin and by your project. Produces 256 bits, shown as 64 hex characters. *Section 3*
Slashing	Destroying a misbehaving participant's deposit as punishment. Notably **absent** from Algorand. *Section 12*
Sybil attack	Gaining disproportionate influence by creating many fake identities. *Section 10*
Throughput	How much the system can process per unit time — transactions per second, or megabytes per hour. *Section 11*

SECTION 15 • HANDS-ON

Build a blockchain in 60 lines

Reading about hash chains and building one are different kinds of knowing. This needs nothing installed — Python 3 is already on your machine.

Save the three parts below into one file, `toychain.py`, then run `python3 toychain.py`.

Part 1 — a block, and how it hashes itself

```
import hashlib, json, time

def sha(data: str) -> str:
    return hashlib.sha256(data.encode()).hexdigest()

class Block:
    def __init__(self, index, transactions, prev_hash):
        self.index        = index
        self.timestamp    = time.time()
        self.transactions = transactions
        self.prev_hash    = prev_hash

    def hash(self) -> str:
        # the hash must cover EVERY field, or a field could be
        # edited without detection. sort_keys makes it deterministic.
        payload = json.dumps({
            'index':        self.index,
            'timestamp':    self.timestamp,
            'transactions': self.transactions,
            'prev_hash':    self.prev_hash,
        }, sort_keys=True)
        return sha(payload)
```

The only subtle line is `sort_keys=True`. Dictionaries have no guaranteed order, so without it the same block could serialise two different ways and produce two different hashes — and validation would fail at random. Note also that the hash covers **every** field: leave one out and that field could be edited undetected.

Part 2 — the chain, and the validation walk

```
class Chain:
    def __init__(self):
        # the genesis block: the only one with no predecessor
        self.blocks = [Block(0, ['genesis'], '0'*64)]

    def add(self, transactions):
        prev = self.blocks[-1]
        self.blocks.append(Block(len(self.blocks), transactions, prev.hash()))

    def validate(self) -> bool:
        for i in range(1, len(self.blocks)):
            cur, prev = self.blocks[i], self.blocks[i-1]
            if cur.prev_hash != prev.hash():
                print(f'  BROKEN at block {i}:')
                print(f'    block {i} stores prev_hash {cur.prev_hash[:16]}...')
                print(f'    block {i-1} actually hashes to {prev.hash()[:16]}...')
                return False
        return True
```

`validate()` is the whole idea of section 6 in six lines: walk the chain, and at each step check that what a block *claims* its predecessor hashed to still matches what that predecessor *actually* hashes to now.

Part 3 — run it, then break it

```
# ---- try it --------------------------------------------------
c = Chain()
c.add(['swayam pays alice 500'])
c.add(['alice pays bob 200'])
c.add(['bob pays carol 50'])

print('intact chain valid? ', c.validate())

# now tamper with block 1 and watch the damage propagate
c.blocks[1].transactions = ['swayam pays alice 5000000']
print('after tampering:    ', c.validate())
```

What you should see

```
intact chain valid?  True
  BROKEN at block 2:
    block 2 stores prev_hash 3f8a1c04b2e97d55...
    block 1 actually hashes to c710d9e4a8f30126...
after tampering:     False
```

READ THE OUTPUT CAREFULLY — THIS IS SECTION 6 MADE REAL

You edited block **1**, but the error is reported at block **2**. That is the cascade: block 2 is the first place the damage becomes *visible*, because block 2 is what stores block 1's old hash.

Now try to repair it. Set `c.blocks[2].prev_hash = c.blocks[1].hash()` and validate again. It will fail at block 3 instead. Repair that, and it fails at block 4. **You have to redo the entire chain.**

Three extensions, if you want to go further

1. **Add signatures.** Use `cryptography`'s Ed25519 so each transaction is signed, and have `validate()` reject unsigned ones. This makes section 4 concrete.
2. **Add proof of work.** Give `Block` a `nonce` and loop until the hash starts with four zeros. Time it, then try five zeros. You will feel section 11 in your laptop fan.
3. **Make two chains disagree.** Build two `Chain` objects, add different blocks to each, and write the rule that picks a winner. You have just written fork resolution.

SECTION 16

Checkpoint

Answer these out loud, without notes, before moving to Phase 2. The section to revisit is named beside each one. Model answers follow — resist reading them first.

01	Why is a bank able to prevent double spending trivially, while a decentralised network cannot?	§2
02	What are the four properties of a hash function, and which one makes mining possible?	§3
03	A signature proves who authorised a transaction. Name precisely what it does **not** prove, and why that gap matters.	§4, 5
04	You edit a transaction in block 10 of a 20-block chain. Describe exactly what breaks, and what an attacker would have to do to hide it.	§6
05	Why is a Byzantine fault harder to tolerate than a crash fault? Give a concrete example where signatures do not help.	§9
06	Explain the reasoning behind `n > 3f` — why two thirds and not one half?	§9
07	What is a Sybil attack, and how do proof of work and proof of stake each defeat it? Note they defeat it in genuinely different ways.	§10, 11, 12
08	Why does Bitcoin make you wait roughly an hour? Name the two separate design decisions that add up to that number.	§11
09	What is the difference between probabilistic and absolute finality, and which does Algorand provide?	§11, 12
10	Hyperledger Fabric is permissioned, so every member is known and vetted. Why does it still need Byzantine fault tolerance? (This is the justification for your entire project.)	§13

SECTION 16 • MODEL ANSWERS

Answers

If your answer covered the same ground in your own words, you have it. Exact wording does not matter; the mechanism does.

01 The bank is a single point of decision: it sees both transactions and processes whichever arrived first. A decentralised network has no single observer, so two halves can each legitimately believe they saw the first one. Nothing inside a signed transaction records when it was created relative to another.

02 Deterministic, fixed size, one-way, avalanche. **One-way** makes mining possible: since you cannot invert the function, the only way to find a nonce producing a small hash is to guess, which is what makes the work real.

03 It does not prove **ordering** — when this transaction happened relative to any other. I can sign two conflicting transactions and both signatures verify perfectly. Authenticity and ordering are separate problems, and only the second needs consensus.

04 Editing block 10 changes its hash completely (avalanche). Block 11 stores block 10's old hash, so block 11 no longer matches. Repairing block 11 changes block 11's hash, breaking block 12, and so on. To hide it the attacker must recompute every block from 10 to 20 — and in proof of work, redo all of that mining faster than the honest network extends the real chain.

05 A crashed node is silent, so everyone still sees a consistent (if incomplete) world. A Byzantine node tells different peers different things, so honest nodes end up confidently believing contradictory facts. Signatures do not help because the liar signs both messages honestly — each is genuinely from them. The signature proves authorship, not consistency.

06 You must be able to decide without hearing from `f` nodes, since they may be silent — so you act on `n - f` replies. Among those, up to `f` may be lies. For the honest replies to still outnumber the lies you need `(n - f) - f > f`, that is `n > 3f`. One half is not enough because a half-sized honest majority can be exactly cancelled by liars plus the unreachable.

07 A Sybil attack is one party gaining outsized influence by posing as many. Proof of work defeats it **physically** — a million identities on one machine still have that machine's hash rate. Proof of stake defeats it **arithmetically** — splitting a balance across accounts leaves the total unchanged. Neither makes identities scarce; both make identities irrelevant.

08 Two decisions compound. First, blocks are deliberately slow (ten minutes) so that simultaneous wins, and therefore forks, stay rare. Second, because forks still happen, you wait about six blocks for the branch to be settled. Six times ten minutes is roughly an hour.

09 Probabilistic finality means a transaction becomes exponentially *unlikely* to be reversed as blocks pile on, but never certain — that is Bitcoin. Absolute finality means reversal is mathematically impossible once committed — that is Algorand, and it is the central claim of the paper you are implementing.

10 Known does not mean trustworthy. A vetted member can still be compromised, coerced, or simply buggy — identity tells you who misbehaved, it does not prevent it. Fabric 2.5 ships only Raft, which tolerates nodes that **crash** but not nodes that **lie**; Solo and Kafka are deprecated and no BFT option exists. So Fabric 2.5 has no Byzantine-tolerant ordering service at all — which is the gap your project fills.

YOU ARE DONE WITH PHASE 1

Next is **Phase 2 — Cryptography and VRFs**, about ten hours. It goes deeper on hashing and signatures, then introduces the Verifiable Random Function, which is the single idea Algorand is built on.

If you would rather keep momentum on the tooling side instead, the Homebrew and Go installs are still outstanding, and Phase 5 can start the moment they finish.